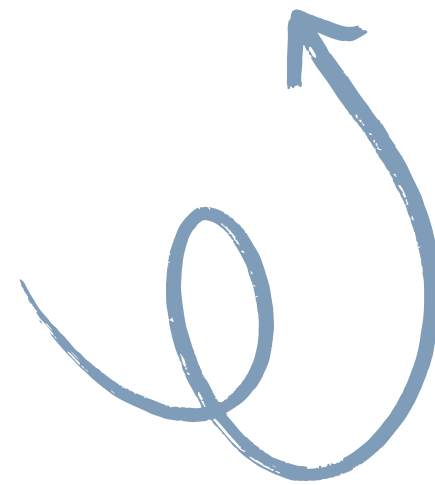


Mise en place d'une communication sécurisée avec mTLS et SSL



présenté par:
Ousmane Diallo

M. Maodo Diop



Introduction et Contexte



Présentation du projet

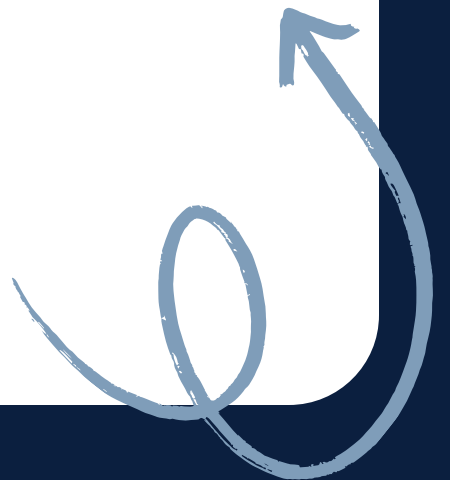
Objectifs d'implémentation de mTLS et SSL

Introduction au projet

Ce projet vise à **implémenter mTLS** et SSL pour sécuriser les communications entre le backend et le frontend, garantissant ainsi la confidentialité et l'intégrité des données échangées.

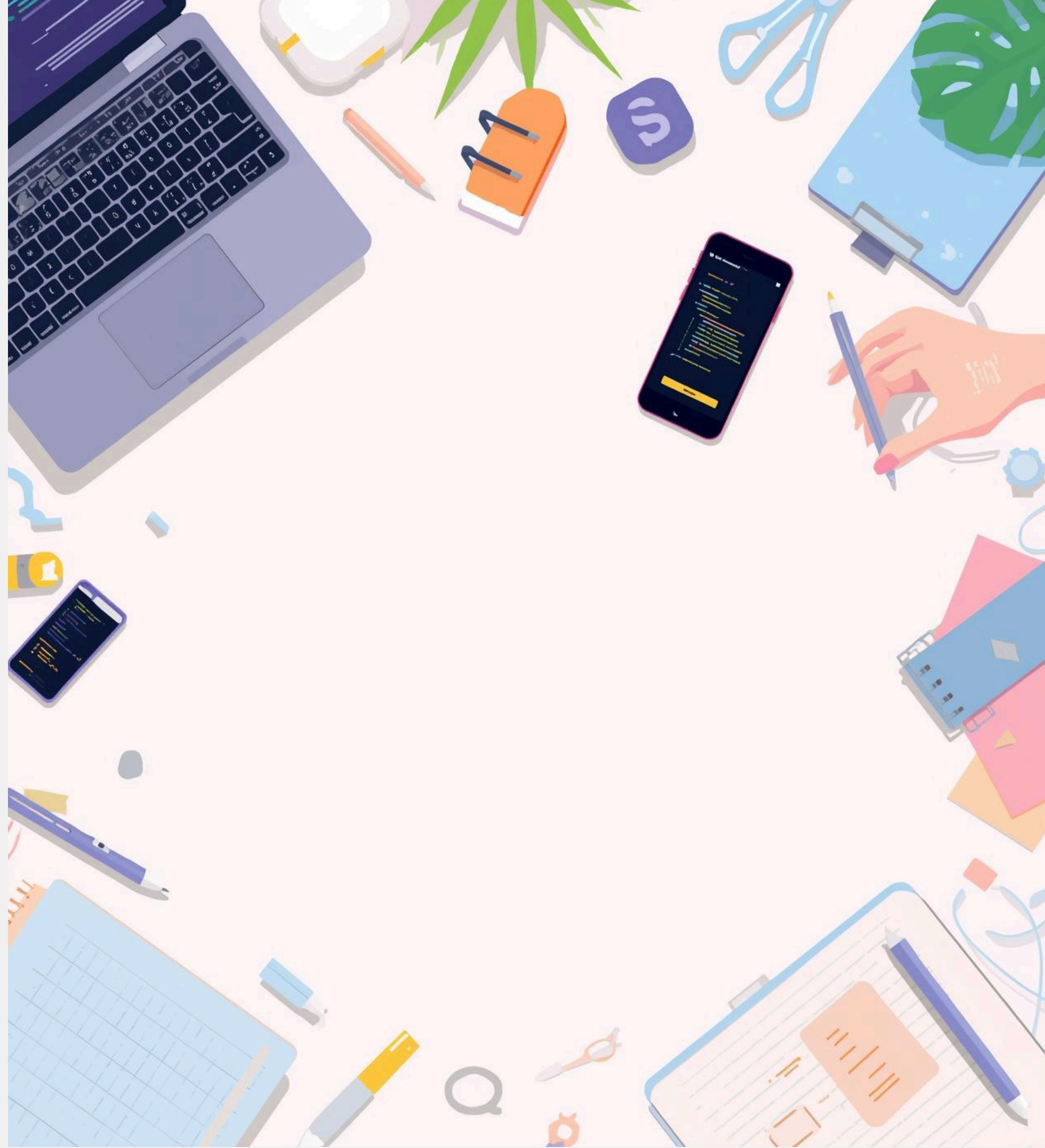
Importance de la sécurité

La sécurisation des communications avec SSL et mTLS est cruciale pour protéger les données sensibles contre les attaques, **renforçant** la confiance des utilisateurs dans l'application.



Objectifs pédagogiques et outils

L'objectif principal est de **maîtriser la configuration** de mTLS et SSL, tout en découvrant les outils essentiels comme keytool, openssl et curl.exe pour tester et sécuriser les communications entre backend et frontend.



Concepts de SSL et mTLS



Fondamentaux de SSL et mTLS

Concepts clés pour sécuriser les communications

Explication de SSL

SSL, ou Secure Sockets Layer, est un protocole qui permet de chiffrer les communications entre un client et un serveur, garantissant la confidentialité et l'intégrité des données échangées.

mTLS

mTLS, ou mutual TLS, renforce la sécurité en exigeant que le client présente un certificat valide au serveur, établissant ainsi une authentification mutuelle qui sécurise davantage les échanges.

Différence avec SSL

Contrairement à SSL standard, qui n'authentifie que le serveur, mTLS nécessite une vérification du client, offrant ainsi une couche de sécurité supplémentaire, essentielle pour les applications sensibles.

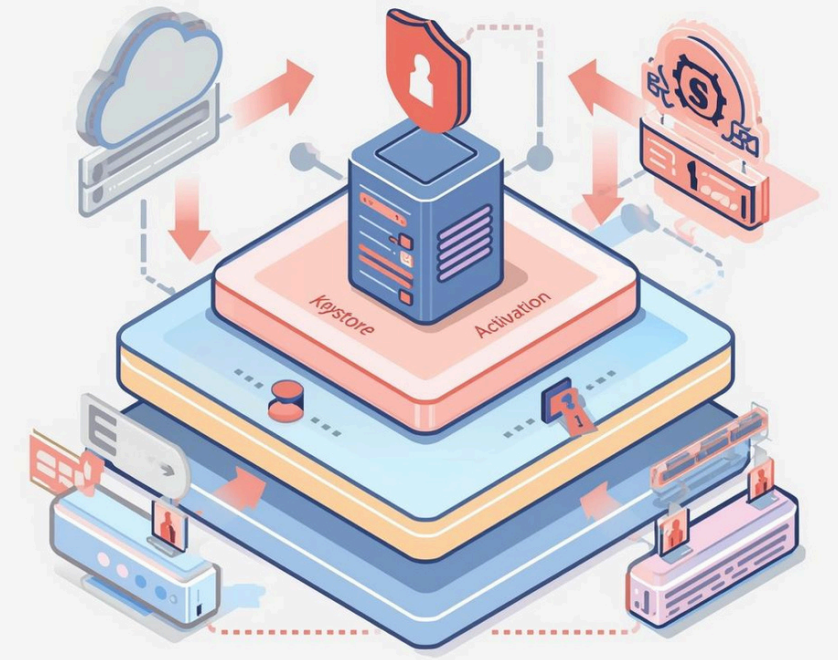


Configuration mtls et ssl sur Backend Spring Boot



Activation de SSL dans Spring Boot

1. Générer un certificat SSL (keystore)
2. Déplacer le keystore dans ton projet Spring Boot
3. Configurer Spring Boot pour activer HTTPS



1. Générer un certificat SSL (keystore)

```
PS C:\Users\DELL> keytool -genkeypair -alias server-cert -keyalg RSA -keysize 2048 -validity 365 -keystore server-keystore.p12 -storetype PKCS12
Enter keystore password:
Keystore password is too short - must be at least 6 characters
Enter keystore password:
Re-enter new password:
Enter the distinguished name. Provide a single dot (.) to leave a sub-component empty or press ENTER to use the default value in braces.
What is your first and last name?
What is the name of your organizational unit?
What is the name of your organization?
What is the name of your City or Locality?
What is the name of your State or Province?
What is the two-letter country code for this unit?
Is CN=ousmane diallo, OU=Unknown, O=Unknown, L=bambey, ST=bambey, C=sn correct?
[no]: yes
Generating 2048-bit RSA key pair and self-signed certificate (SHA384withRSA) with a validity of 365 days
        for: CN=ousmane diallo, OU=Unknown, O=Unknown, L=bambey, ST=bambey, C=sn
PS C:\Users\DELL>
```

2. Déplacer le keystore dans ton projet Spring Boot

 static	21/08/2025 21:52	Dossier de fichiers	
 templates	21/08/2025 21:52	Dossier de fichiers	
 application	06/09/2025 21:52	Fichier source Proper...	1 Ko
☒  server-keystore	14/11/2025 12:26	Échange d'informatio...	3 Ko



3. Configurer Spring Boot pour activer HTTPS

```
24
25 # SSL Configuration
26 server.port=8443
27 server.ssl.enabled=true
28 server.ssl.key-store=classpath:server-keystore.p12
29 server.ssl.key-store-password=changeit
30 server.ssl.key-store-type=PKCS12
31
32
```



Activation de mTL dans Spring Boot

1. Générer le certificat CLIENT auto-signé
2. Exporter le certificat public du client
3. Créer un truststore contenant ce certificat client

1. Générer le certificat CLIENT auto-signé

```
Windows PowerShell
PS C:\Users\DELL> keytool -genkeypair -alias client-cert -keyalg RSA -keysize 2048 -validity 365 -keystore client-keystore.p12 -storetype PKCS12 -storepass changeit -keypass changeit
Enter the distinguished name. Provide a single dot (.) to leave a sub-component empty or press ENTER to use the default value in braces.
What is your first and last name?
[Unknown]: ousmane diallo
What is the name of your organizational unit?
[Unknown]:
What is the name of your organization?
[Unknown]:
What is the name of your City or Locality?
[Unknown]: bambey
What is the name of your State or Province?
[Unknown]: bambey
What is the two-letter country code for this unit?
[Unknown]: sn
Is CN=ousmane diallo, OU=Unknown, O=Unknown, L=bambey, ST=bambey, C=sn correct?
[no]: yes

Generating 2048-bit RSA key pair and self-signed certificate (SHA384withRSA) with a validity of 365 days
for: CN=ousmane diallo, OU=Unknown, O=Unknown, L=bambey, ST=bambey, C=sn
PS C:\Users\DELL> |
```


2. Exporter le certificat public du client

```
PS C:\Users\DELL> keytool -export -alias client-cert -file client-cert.cer -keystore client-keystore.p12
Enter keystore password:
Certificate stored in file <client-cert.cer>
PS C:\Users\DELL>
```

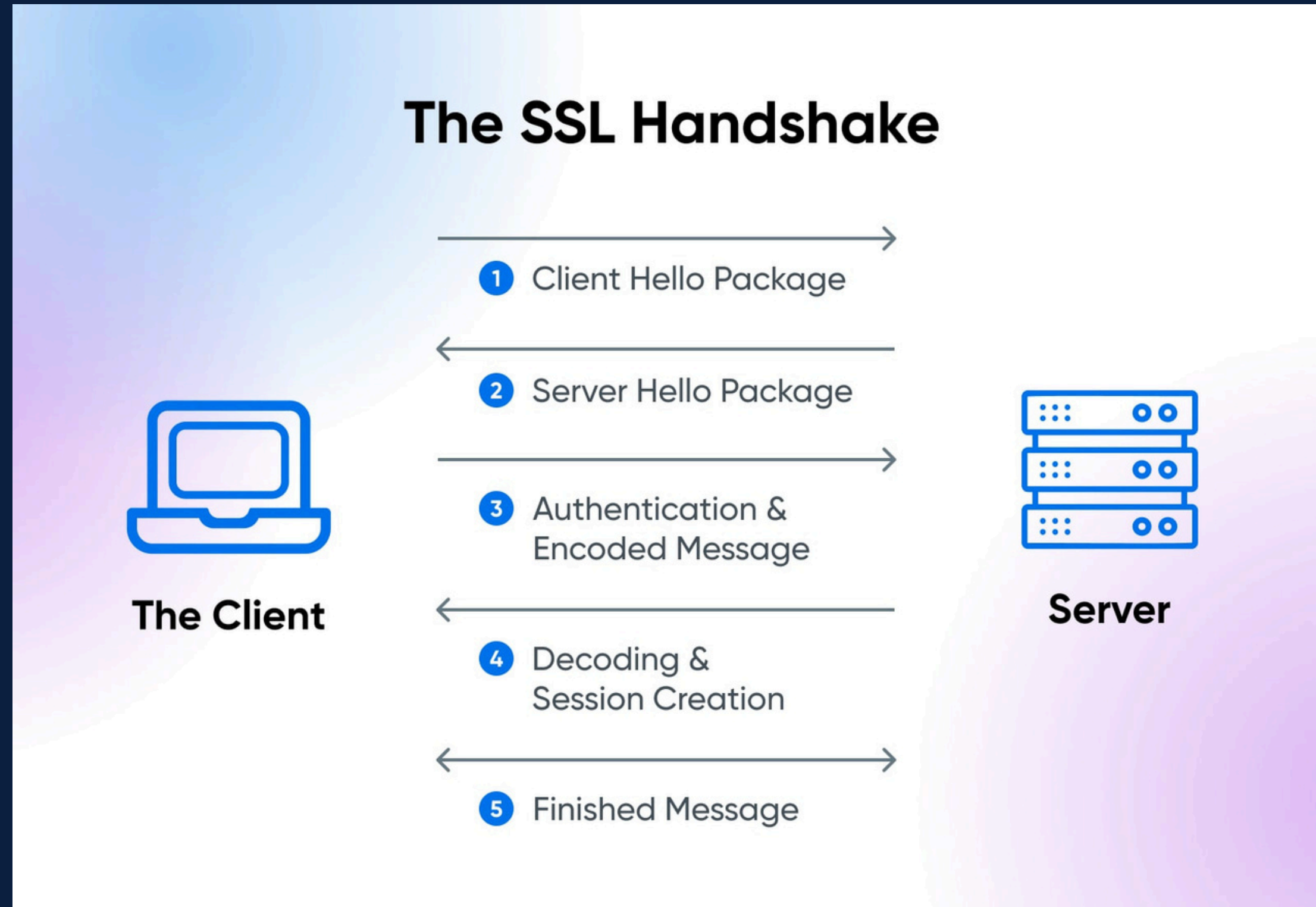
3. Créer un truststore contenant ce certificat client

```
PS C:\Users\DELL> keytool -import -alias client-cert -file client-cert.cer -keystore server-truststore.p12 -storetype PKCS12
Enter keystore password:
Re-enter new password:
Owner: CN=ousmane diallo, OU=Unknown, O=Unknown, L=bambey, ST=bambey, C=sn
Issuer: CN=ousmane diallo, OU=Unknown, O=Unknown, L=bambey, ST=bambey, C=sn
Serial number: 385237905f9c0163
Valid from: Fri Nov 14 16:04:00 GMT 2025 until: Sat Nov 14 16:04:00 GMT 2026
Certificate fingerprints:
    SHA1: A4:95:9B:3E:E5:9A:FD:FC:9B:69:02:A3:FB:E9:56:39:47:C6:F0:9F
    SHA256: 3A:A5:46:B5:AE:57:B0:7C:2B:1D:71:6B:81:55:B7:FD:D3:07:E9:3D:3D:47:E5:50:5F:5C:28:3D:B0:9C:3D:1B
Signature algorithm name: SHA384withRSA
Subject Public Key Algorithm: 2048-bit RSA key
Version: 3

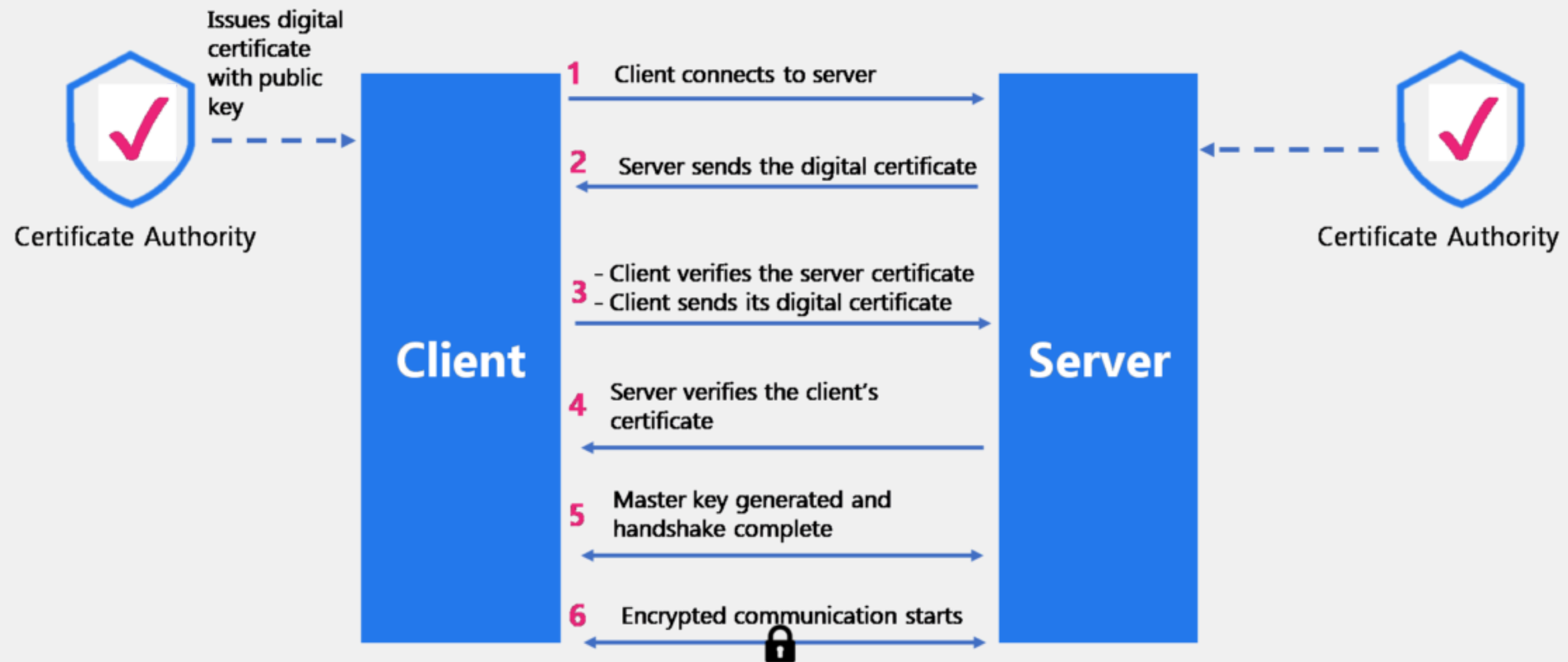
Extensions:
#1: ObjectId: 2.5.29.14 Criticality=false
SubjectKeyIdentifier [
KeyIdentifier [
0000: 3C 3B 2F 2D 71 1B 3B BD    6C 1D 9E DB FC 5D 29 F2    < ; / - q . ; . l . . . . ] ) .
0010: 95 5F 9C 05                . _ . .
]
]

Certificate was added to keystore
PS C:\Users\DELL> |
```

Fonctionnement du SSL / TLS classique (Serveur uniquement)



Fonctionnement du mTLS : Mutual TLS (Client et serveur s'authentifient)



Sécurisation du Frontend

Utilisation d'un Proxy Node.js

Raisons d'utiliser

L'utilisation d'un proxy Node.js permet de centraliser la gestion des certificats SSL, de simplifier la configuration du frontend et d'assurer une communication sécurisée entre Angular et le backend.

Configuration du serveur

La configuration du serveur proxy Node.js implique l'installation des certificats SSL, la mise en place des redirections HTTPS, et la gestion des requêtes pour garantir la sécurité des données.

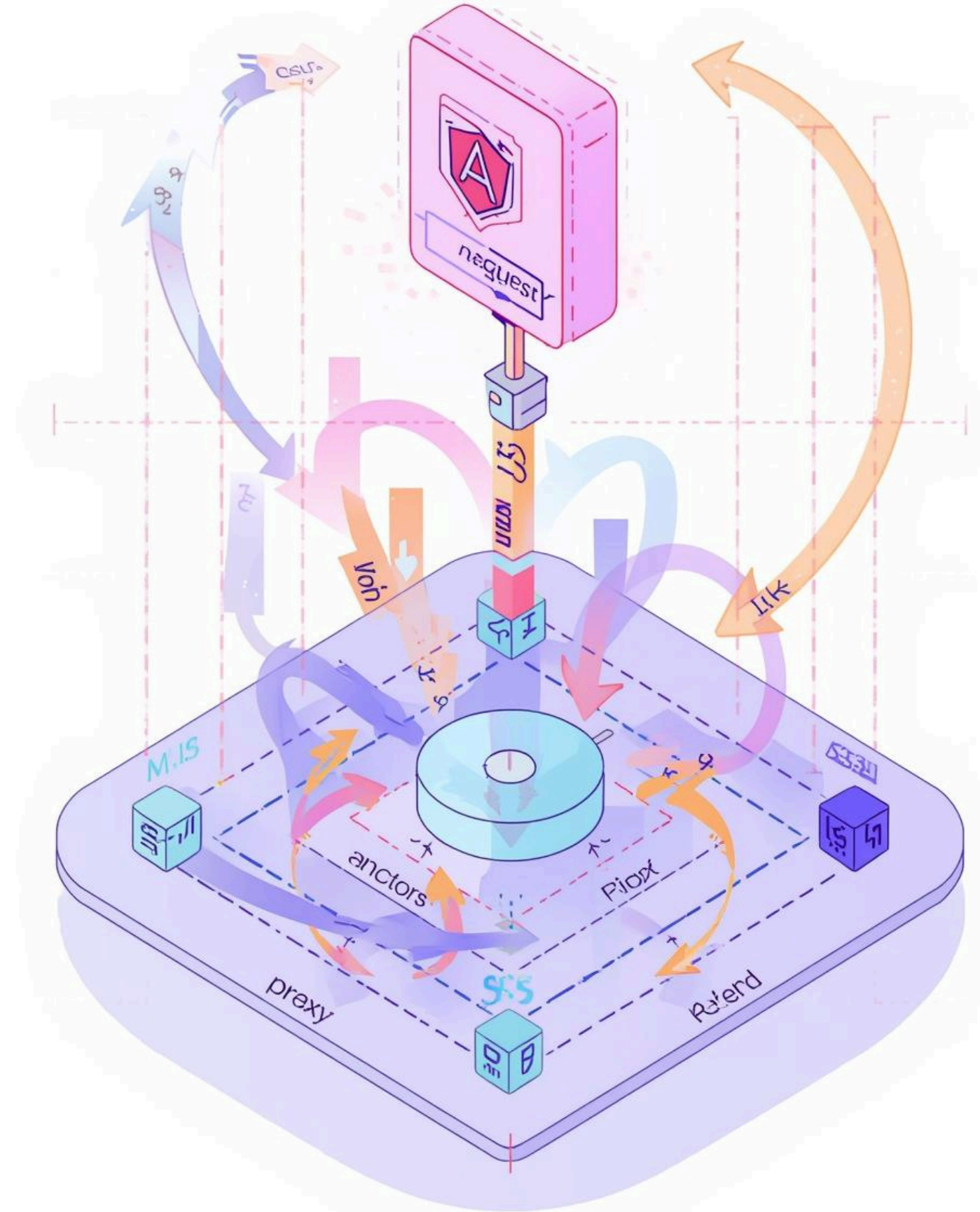
Paramétrage Angular

Le paramétrage côté Angular nécessite d'ajuster les appels API pour passer par le proxy, garantissant ainsi que toutes les communications utilisent HTTPS et respectent les exigences de sécurité.



configuration du ssl sur le Front

1. création du certificat
2. la mise en place du serveur proxy



1. création de certificat

```
D:\coursM2\cours deve avancer\projetT1s\projet-ecommerce\Front_Ecommerce>mkdir ssl
```

D:\coursM2\cours deve avancer\projetTls\projet-ecommerce\Front Ecommerce>cd ssl

```
D:\coursM2\cours deve avancer\projetTls\projet-ecommerce\Front_Ecommerce\ssl>openssl req -x509 -newkey rsa:4096 -nodes -out cert.pem -keyout key.pem -days 365
```

```
DELL@DESKTOP-RORQF9H MINGW64 /d/coursM2/cours deve avancer/projetTls/projet-ecommerce/Front_Ecommerce/ssl (master)
```

```
$ openssl req -x509 -newkey rsa:4096 -nodes -out cert.pem -keyout key.pem -days 365
```

```
You are about to be asked to enter information that will be incorporated
into your certificate request.
What you are about to enter is what is called a Distinguished Name or a DN.
There are quite a few fields but you can leave some blank
For some fields there will be a default value,
If you enter '.', the field will be left blank.
```

```
Country Name (2 letter code) [AU]:senegal
String too long, must be at most 2 bytes long
Country Name (2 letter code) [AU]:SN
```



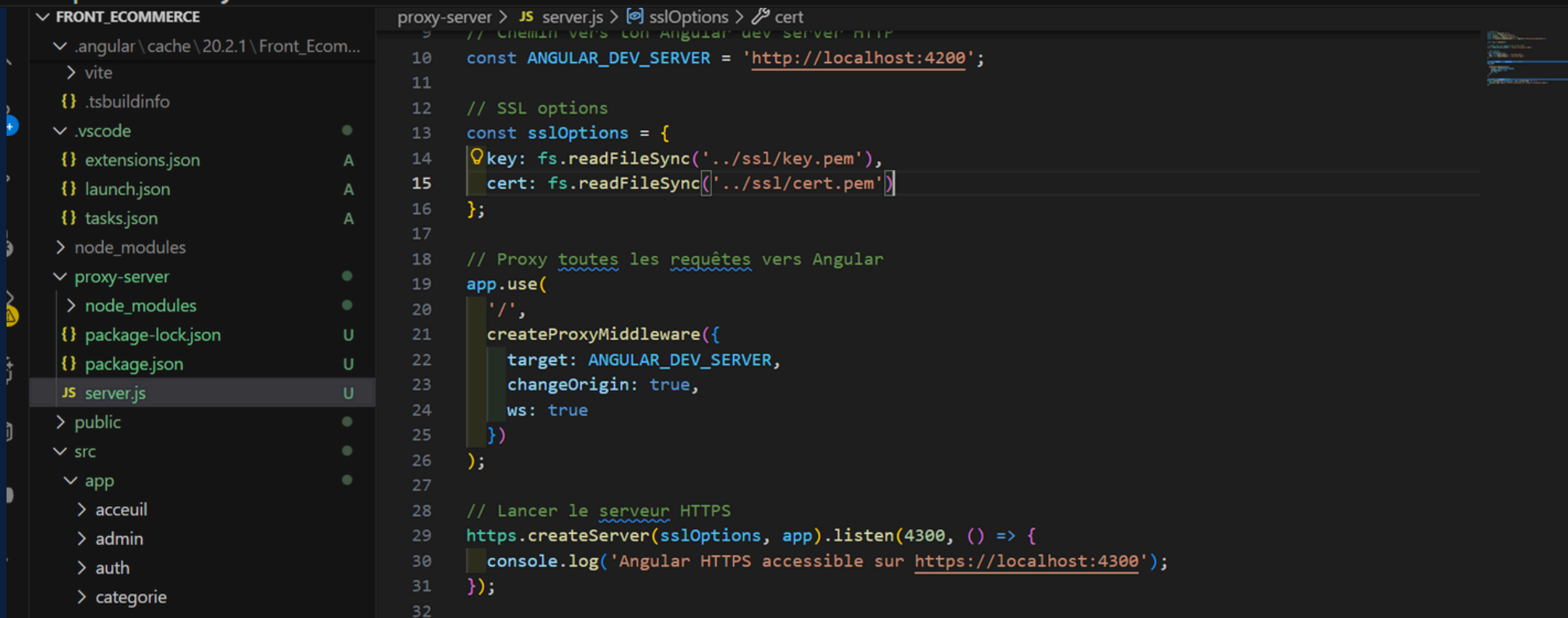
2. la mise en place du serveur proxy

```
D:\coursM2\cours deve avancer\projetTls\projet-ecommerce\Front_Ecommerce>mkdir proxy-server
```

```
D:\coursM2\cours deve avancer\projetTls\projet-ecommerce\Front_Ecommerce>cd proxy-server
```

```
D:\coursM2\cours deve avancer\projetTls\projet-ecommerce\Front_Ecommerce\proxy-server>npm init -y  
Wrote to D:\coursM2\cours deve avancer\projetTls\projet-ecommerce\Front_Ecommerce\proxy-server\package.json:
```

```
{  
  "name": "proxy-server",  
  "version": "1.0.0",  
  "description": "",  
  "main": "server.js",  
  "scripts": {  
    "start": "node server.js"  
  },  
  "keywords": [],  
  "author": "",  
  "license": "ISC"  
}
```



```
proxy-server > JS server.js > sslOptions > cert  
10 // Chemin vers ton Angular dev server HTTP  
11 const ANGULAR_DEV_SERVER = 'http://localhost:4200';  
12  
13 // SSL options  
14 const sslOptions = {  
15   key: fs.readFileSync('../ssl/key.pem'),  
16   cert: fs.readFileSync('../ssl/cert.pem')  
17 };  
18  
19 // Proxy toutes les requêtes vers Angular  
20 app.use(  
21   '/',  
22   createProxyMiddleware({  
23     target: ANGULAR_DEV_SERVER,  
24     changeOrigin: true,  
25     ws: true  
26   })  
27 );  
28  
29 // Lancer le serveur HTTPS  
30 https.createServer(sslOptions, app).listen(4300, () => {  
31   console.log('Angular HTTPS accessible sur https://localhost:4300');  
32 });
```

Conseils owasp

Toujours utiliser TLS (HTTPS) partout

Désactiver TLS obsolètes

Utiliser des certificats fiables

Protéger la clé privée



Démonstration



Conclusion

SSL/TLS sécurise les communications client-serveur en assurant confidentialité, intégrité et authenticité.

Le mTLS ajoute une authentification mutuelle, garantissant que serveur et client sont bien identifiés.

Les clés de session chiffrent les échanges, protégeant les données contre l'interception ou la modification.

L'utilisation correcte des certificats réduit les risques de vulnérabilités liées aux mauvaises configurations.



Sécurité Renforcée



Merci de votre attention